

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of a Convolutional Neural Network (CNN) for Image Classification

Name	Vinay Sathish
Roll Number	24011101126
Degree & Branch	B.Tech Artificial Intelligence & Data Science (Section-B)
Subject Code & Name	CS3807 – Deep Learning Laboratory

1. Objective

Implement a Convolutional Neural Network (CNN) using TensorFlow/Keras for multi-class image classification on the CIFAR-10 dataset. Learn convolution and pooling operations, CNN architecture design, model training and evaluation, and visualization of learned convolutional feature maps.

2. Learning Outcomes

Should be able to:

1. Explain the architecture and working of a Convolutional Neural Network.
2. Preprocess RGB image datasets for CNN input.
3. Build, compile, and train a CNN using Keras.
4. Evaluate classification performance using standard metrics.
5. Visualize and interpret learned convolutional feature maps.

3. Background Theory

A Convolutional Neural Network (CNN) is a deep learning architecture designed for grid-structured data such as images. Unlike a fully connected MLP, a CNN uses learnable convolutional filters that slide over the input to extract local spatial features (edges, textures, shapes), followed by pooling layers that reduce spatial dimensionality while retaining the most salient features.

Convolution operation:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

where I is the input feature map and K is the convolutional kernel.

Max Pooling reduces each spatial region to its maximum activation, providing translation invariance and reducing the number of parameters in subsequent layers.

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss (Sparse Categorical Cross-Entropy):

$$L = -\log(\hat{y}_{y_{\text{true}}})$$

Architecture used in this experiment:

Conv2D(32, 3×3, ReLU) → MaxPool(2×2) → Conv2D(64, 3×3, ReLU) → MaxPool(2×2) →
Conv2D(128, 3×3, ReLU) → Flatten → Dense(128, ReLU) → Dropout(0.5) → Dense(10, Softmax)

4. Dataset

Dataset: CIFAR-10

Training Images: 50,000

Testing Images: 10,000

Classes: 10 (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)

Image Size: $32 \times 32 \times 3$ (RGB)

Result: The dataset was loaded via `tf.keras.datasets.cifar10`. The loaded shapes matched specification exactly: training set (50000, 32, 32, 3) and testing set (10000, 32, 32, 3).

5. Data Preprocessing

1. Load CIFAR-10.
2. Display sample images with class labels.
3. Plot class distribution.
4. Normalize pixel values to $[0, 1]$.

Result: Pixel values were cast to `float32` and divided by 255, mapping the raw $[0, 255]$ integer range to $[0, 1]$. No flattening was required, since Conv2D layers operate directly on the 3-dimensional (32, 32, 3) image tensors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load CIFAR-10.
- Print dataset dimensions.
- Display ten sample images with labels.
- Plot class distribution.

Result: Training Images: (50000, 32, 32, 3); Testing Images: (10000, 32, 32, 3). Sample images and the class distribution plot are provided in Section 8.

Task 2: Model Construction

Construct a CNN with

- Conv2D(32 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(64 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(128 filters, 3×3 , ReLU, same padding)
- Flatten $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10, Softmax)

Result:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 128)	1,048,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

- Total params: 1,143,242 (4.36 MB)
- Trainable params: 1,143,242 (4.36 MB)
- Non-trainable params: 0 (0.00 B)

Task 3: Model Training

Compile using

- Optimizer: Adam
- Loss: Sparse Categorical Cross-Entropy
- Metric: Accuracy

Train for 10 epochs with batch size 64, using a 20% validation split.

Result: Training accuracy rose from 37.53% (epoch 1) to 78.01% (epoch 10), while validation accuracy rose from 51.40% to 73.58% over the same period. The epoch-wise training log is summarized below.

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss
1	0.3753	1.6973	0.5140	1.3407
2	0.5245	1.3247	0.6113	1.0913
3	0.5909	1.1522	0.6466	0.9904
4	0.6377	1.0334	0.6854	0.8973
5	0.6702	0.9386	0.6975	0.8535
6	0.6962	0.8681	0.7189	0.8083
7	0.7212	0.7896	0.7142	0.8358
8	0.7488	0.7169	0.7260	0.8158
9	0.7646	0.6727	0.7265	0.8183
10	0.7801	0.6148	0.7358	0.7831

Full accuracy/loss curves are provided in Section 8.

Task 4: Model Evaluation

Compute

- Test Accuracy and Loss
- Precision, Recall, F1-score (per class and averaged)
- Confusion Matrix
- Classification Report

Result: On the held-out test set (10,000 images):

Metric	Value
Test Accuracy	0.7345
Test Loss	0.8070
Precision (macro)	0.74
Recall (macro)	0.73
F1-score (macro)	0.74

Classification Report:

Class	Precision	Recall	F1-score	Support
Airplane	0.79	0.77	0.78	1000
Automobile	0.88	0.83	0.86	1000
Bird	0.67	0.59	0.63	1000
Cat	0.52	0.58	0.55	1000
Deer	0.70	0.68	0.69	1000
Dog	0.59	0.66	0.62	1000
Frog	0.79	0.79	0.79	1000
Horse	0.77	0.80	0.78	1000
Ship	0.84	0.86	0.85	1000
Truck	0.83	0.79	0.81	1000
Accuracy	0.73			10000
Macro avg	0.74	0.73	0.74	10000
Weighted avg	0.74	0.73	0.74	10000

The **Automobile** and **Ship** classes are the strongest performers (F1-score 0.86 and 0.85), while **Cat** is by far the weakest (F1-score 0.55), consistently confused with visually similar animal classes such as Dog and Bird. The confusion matrix is provided in Section 8.

Task 5: Model Interpretation

- Compute total trainable parameters.
- Visualize convolutional feature maps for a sample test image.
- Display a sample prediction (actual vs. predicted label).

Result: The model has **1,143,242** trainable parameters. Feature maps were extracted from all three convolutional layers for a single test image, and a qualitative prediction example is shown in Section 8.

7. Plots Generated

The following plots were generated during the experiment.

Plot	Purpose	Expected Inference
Sample Images	Dataset Familiarization	Visually confirm image content, resolution, and label correctness.
Class Distribution	Balance Check	Verify all classes are equally represented.
Training vs Validation Accuracy & Loss	Learning Behaviour	Assess convergence and detect overfitting.
Confusion Matrix	Performance Evaluation	Identify which classes are most frequently confused.
Convolutional Feature Maps (per layer)	Model Interpretability	Visualize what spatial features each convolutional layer has learned.
Sample Prediction	Qualitative Evaluation	Spot-check a single prediction against the ground truth label.

7.1 Generated Plots and Inference



Figure 1: Sample CIFAR-10 images

Inference: The ten sample images confirm the dataset consists of low-resolution (32×32) RGB natural images spanning both vehicles (automobile, truck, ship, airplane) and animals (bird, cat, deer, dog, frog, horse), with far more visual complexity and background clutter than a grayscale dataset like Fashion-MNIST.

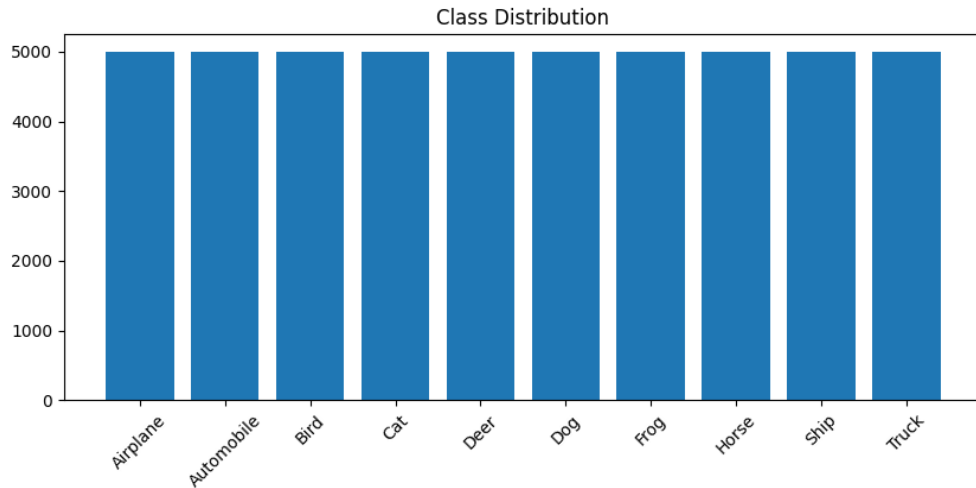


Figure 2: Class distribution (training set)

Inference: All 10 classes are equally represented in the training set (5,000 images each), confirming the dataset is perfectly balanced and that accuracy is a fair evaluation metric without needing class reweighting.

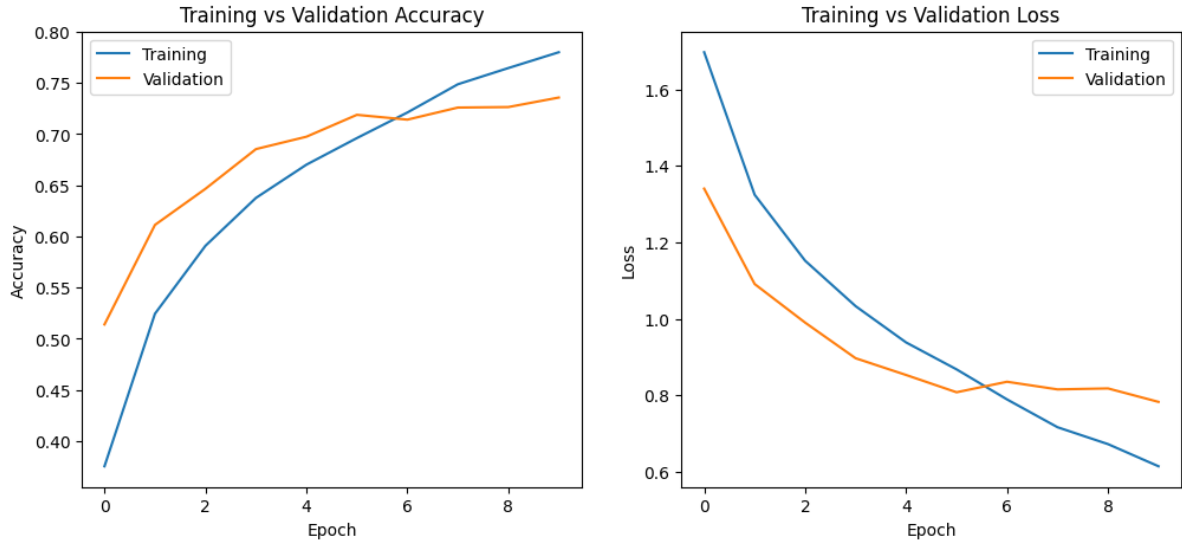


Figure 3: Training vs. validation accuracy and loss

Inference: Training accuracy climbs steadily and consistently through all 10 epochs, while validation accuracy improves quickly at first but grows more slowly after epoch 6 and briefly dips at epoch 7, and validation loss flattens out and even ticks upward around epochs 7 to 9 while training loss keeps falling which could be a sign that the model is starting to overfit and would likely benefit from more epochs of regularization (e.g., data augmentation) or early stopping.

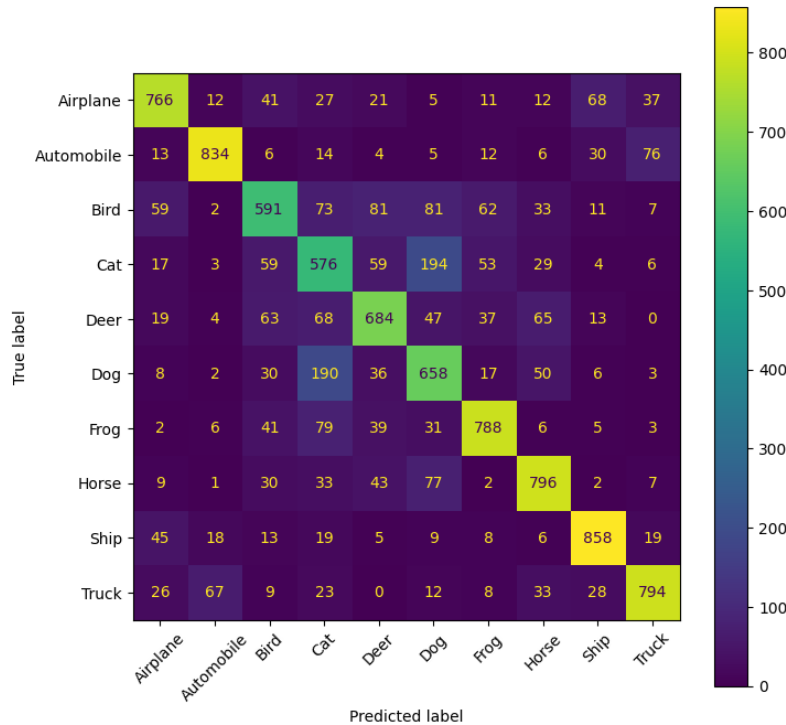


Figure 4: Confusion matrix (test set)

Inference: The strongest diagonal entries belong to Automobile, Ship, and Truck, which have visually distinct shapes and backgrounds, while Cat, Dog, and Bird show substantial mutual confusion which is consistent with these being small, texture-similar animal classes that are hard to distinguish at 32×32 .

resolution.

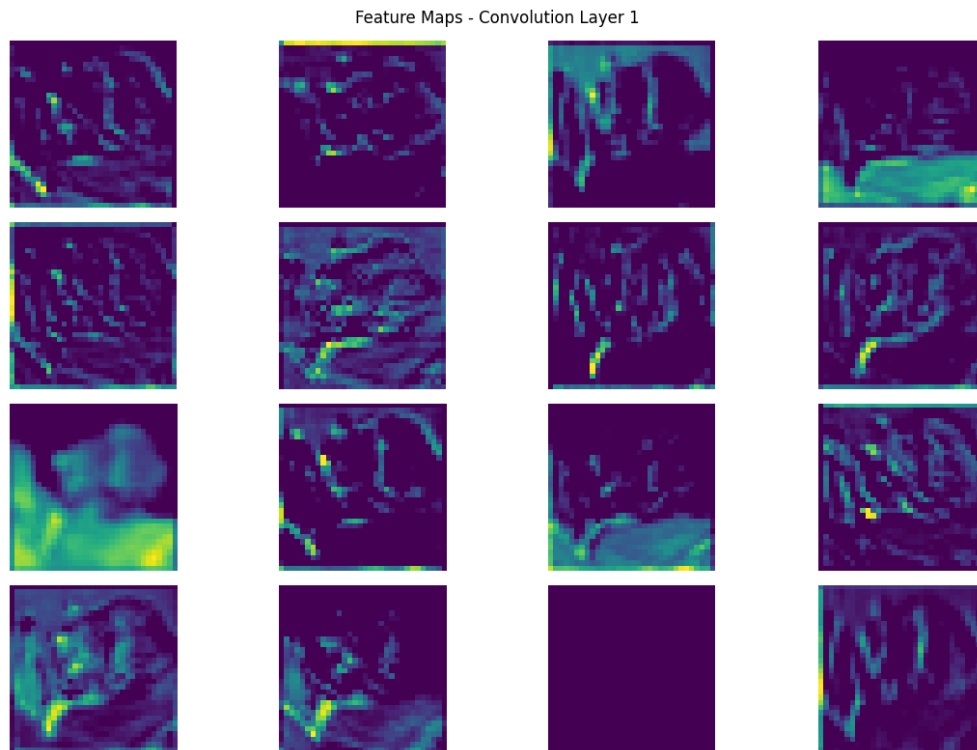


Figure 5: Feature maps – Convolution Layer 1

Inference: The first convolutional layer's 32 filters extract low-level, generic features such as edges, colour contrasts, and coarse object outlines, with most filters still preserving a recognizable silhouette of the input image.

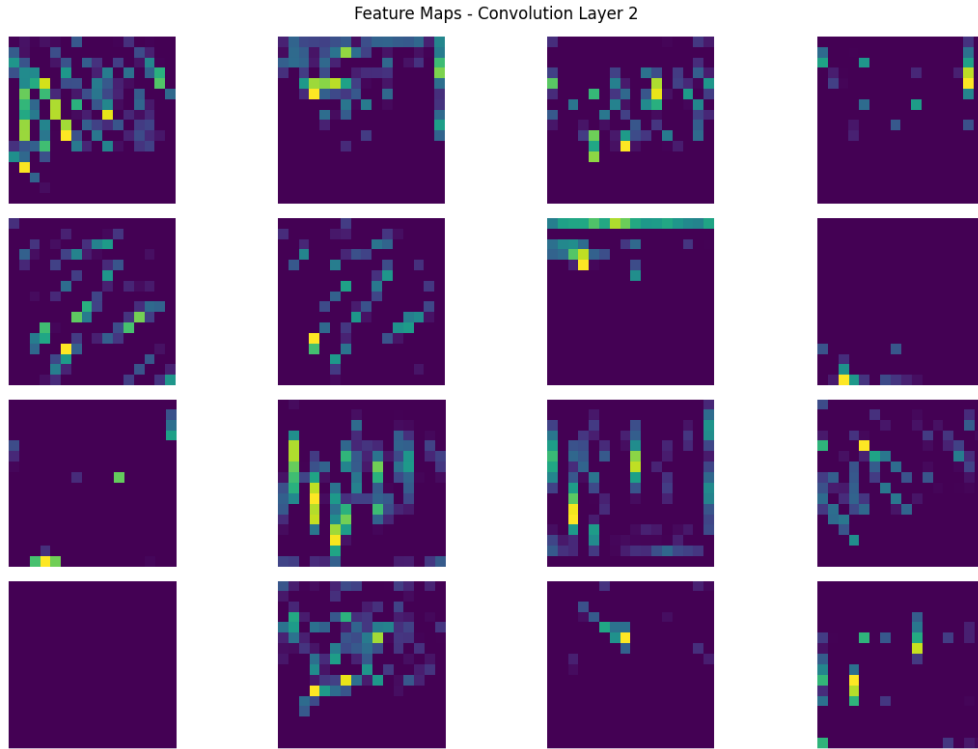


Figure 6: Feature maps – Convolution Layer 2

Inference: By the second convolutional layer, the feature maps are more abstract and spatially coarser (reflecting the intermediate max-pooling step), highlighting specific textured regions rather than the whole object outline.

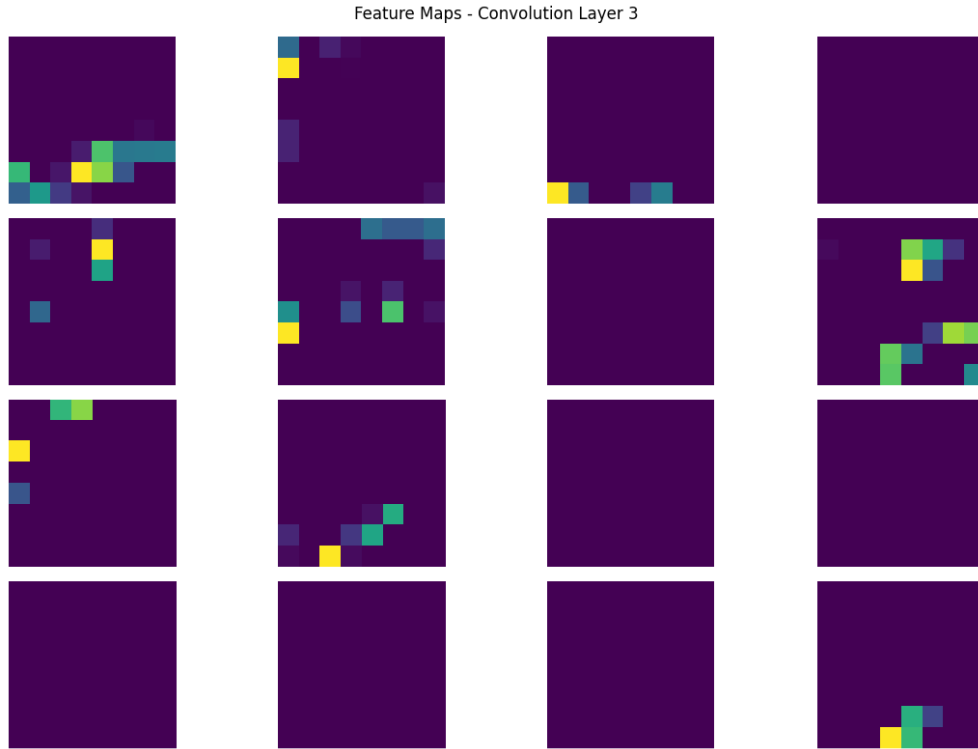


Figure 7: Feature maps – Convolution Layer 3

Inference: The third convolutional layer’s feature maps are the most abstract and spatially compressed of the three, with several filters appearing to activate on highly specific, localized patterns which is consistent with the general CNN principle that deeper layers encode increasingly high-level, task-specific features.



Figure 8: Sample prediction (test image)

Inference: For this sample test image the model correctly predicted “Cat”, matching the ground truth label, despite Cat being the class with the lowest overall F1-score (0.55) which illustrates that the model

succeeds on some individual cat images while still struggling with the class as a whole.

8. Results Summary

Dataset Size	60,000 (50,000 train / 10,000 test)
Image Size	$32 \times 32 \times 3$
Architecture	Conv(32) $\rightarrow$ Pool $\rightarrow$ Conv(64) $\rightarrow$ Pool $\rightarrow$ Conv(128) $\rightarrow$ Dense(128) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(10)
Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Batch Size	64
Epochs	10
Trainable Parameters	1,143,242
Final Training Accuracy	0.7801
Final Validation Accuracy	0.7358
Test Accuracy	0.7345
Test Loss	0.8070
Macro F1-score	0.74
Best-performing Class	Automobile (F1 = 0.86)
Worst-performing Class	Cat (F1 = 0.55)

9. Source Code

The complete source code used for this experiment (dataset loading, preprocessing, CNN construction and training, evaluation, and feature map visualization) is listed below. <https://github.com/vinukohshawenn/deeplearning-neural-network>

10. Conclusion

A Convolutional Neural Network was implemented in TensorFlow/Keras and trained on the CIFAR-10 dataset. The three-block Conv $\rightarrow$ Pool architecture (32 $\rightarrow$ 64 $\rightarrow$ 128 filters) with a Dense(128)+Dropout(0.5) classification head achieved 73.45% test accuracy and a macro F1-score of 0.74 after 10 training epochs, with 1,143,242 trainable parameters. Vehicle classes (Automobile, Ship, Truck) were classified substantially more reliably than small animal classes (Cat, Dog, Bird), which were frequently confused with one another. Training and validation curves showed mild overfitting emerging after roughly epoch 7, suggesting that additional regularization or early stopping could improve generalization in future runs. Feature map visualizations confirmed the expected CNN behaviour of learning progressively more abstract representations with network depth, from simple edges in the first convolutional layer to highly localized, specific patterns in the third.

12. References

1. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, 1998.

2. A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, University of Toronto, 2009. (CIFAR-10 Dataset)
3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
4. TensorFlow/Keras Documentation.